MINISTERUL EDUCAȚIEI, CULTURII ȘI CERCETĂRII
AL REPUBLICII MOLDOVA
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Inginerie Software și Automatică

Canter Daniel - FAF-242

Report

Laboratory work n.3

of Cryptography

Checked by:

Zaica Maia, university assistant

DISA, FCIM, UTM

Chișinău – 2026

1. THEORY

1.1. The Vigenère Cipher

The Vigenère cipher is a polyalphabetic cipher: it works like the Caesar cipher, but instead of one shift it uses several. The key is a word $k = k_1 k_2 ... k_l$, and the i-th letter of the message is shifted by the code of the i-th letter of the key. When the key runs out it starts again from its first letter, so $k_{i+l} = k_i$:

$$c_i = (m_i + k_i) \bmod n, \quad m_i = (c_i - k_i) \bmod n$$

For this lab the alphabet is the Romanian one with $n = 31$ letters, coded with the numbers 0, 1, ..., 30:

A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z

So A = 0, Ă = 1, …, Z = 30. Because the same plaintext letter is shifted differently depending on its position, one letter can become several different letters in the ciphertext, which flattens the frequency distribution that breaks monoalphabetic ciphers.

The key must have at least 7 letters, so even for the shortest allowed key there are

$$31^7 = 27\,512\,614\,111 \approx 2.75 \cdot 10^{10}$$

possible keys, and the number grows with every extra letter. On top of that an attacker does not know the key length.

2. IMPLEMENTATION

The lab is a single Python file, `vigenere.py`, which holds the alphabet, encryption, decryption, the input checks and the menu.

2.1. Letter Encoding

```
ALPHABET = "AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ"
```

The 31 letters are written in alphabetical order, so the code of a letter is just its index in the string. The shift never uses ASCII or Unicode values.

2.2. Encryption and Decryption

```
def encrypt(text, key, alphabet=ALPHABET):
    """c_i = (m_i + k_i) mod n"""
    n = len(alphabet)
    return "".join(
        alphabet[(alphabet.index(m) + alphabet.index(key[i % len(key)])) % n]
        for i, m in enumerate(text)
    )


def decrypt(text, key, alphabet=ALPHABET):
    """m_i = (c_i - k_i) mod n"""
    n = len(alphabet)
    return "".join(
        alphabet[(alphabet.index(c) - alphabet.index(key[i % len(key)])) % n]
        for i, c in enumerate(text)
    )
```

Each letter is turned into its code and shifted by the code of the key letter at the same position. `key[i % len(key)]` repeats the key over the whole text. Python's `%` always returns a value between 0 and $n-1$, so a negative $c_i - k_i$ wraps around without extra code. The alphabet is a parameter, which made it easy to check the functions against the handout example with the 26-letter English alphabet: `PERASPERAADASTRA` with the key `SUPER` gives `HYGEJHYGERVUHXIS`, the same as in the handout.

2.3. Preparing the Text

```
def prepare(text):
    """Remove spaces and uppercase. Returns None if a character is not allowed."""
    text = text.replace(" ", "").upper()
    invalid = [ch for ch in text if ch not in ALPHABET]
    if invalid:
        print(f"Invalid character(s): {' '.join(invalid)}. Allowed: only letters "
              f"of the Romanian alphabet {ALPHABET}, uppercase or lowercase.")
        return None
    return text
```

As the task asks, the spaces are removed first and then all letters are converted to uppercase; Python's `upper()` also handles the Romanian letters (ă → Ă, ș → Ș). Every character that is not in the alphabet is shown in the error message together with the allowed range, and `None` is returned so the input is asked again. The key goes through the same function.

2.4. Input Validation

```
def read_text(prompt):
    while True:
        text = prepare(input(prompt))
        if text is not None:
            return text


def read_key(prompt):
    while True:
        key = read_text(prompt)
        if len(key) >= 7:
            return key
        print(f"Key too short ({len(key)} letters). It must have at least 7 letters.")
```

Every input is asked again until it is valid. The key has to contain only Romanian letters and be at least 7 letters long.

2.5. Program

```
def main():
    while True:
        op = read_choice("\nE - encrypt, D - decrypt: ", ["E", "D"])
        key = read_key("Key (at least 7 letters): ")
        if op == "E":
            print("Ciphertext:", encrypt(read_text("Message: "), key))
        else:
            print("Message:", decrypt(read_text("Ciphertext: "), key))
```

The program reads the operation, the key and then the message or the ciphertext, and prints the result.

3. RESULTS

3.1. Encryption

```
E - encrypt, D - decrypt: X
Invalid choice. Allowed: E, D.

E - encrypt, D - decrypt: e
Key (at least 7 letters): cripto
Key too short (6 letters). It must have at least 7 letters.
Key (at least 7 letters): furtună7
Invalid character(s): 7. Allowed: only letters of the Romanian alphabet
AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ, uppercase or lowercase.
Key (at least 7 letters): Furtună
Message: Știința și arta, 2026
Invalid character(s): , 2 0 2 6. Allowed: only letters of the Romanian alphabet
AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ, uppercase or lowercase.
Message: Știința și arta
Ciphertext: YOZÂIHĂYCRJON
```

The wrong operation, the short key and the invalid characters are all rejected and asked again. Checking the first letters by hand: Ș (22) + F (7) = 29, which is Y, and I (10) + T (23) = 33 mod 31 = 2, which is Â. The two I letters of ȘTIINȚA become Z and Â, because they are shifted by different key letters.

3.2. Decryption

```
E - encrypt, D - decrypt: d
Key (at least 7 letters): furtună
Ciphertext: yozâ ihă ycr jon
Message: ȘTIINȚAȘIARTA
```

The ciphertext is typed in lowercase and with spaces, and it still decrypts to the original message without spaces. Adding the spaces back is done by hand.

4. CONCLUSION

The Vigenère cipher over the Romanian alphabet encrypts and decrypts correctly, and it gives the same result as the handout example when run on the English alphabet. The results show its main difference from the Caesar cipher: the same plaintext letter turns into different ciphertext letters depending on its position, so the letter frequencies are spread out and simple frequency analysis does not work directly.

The main takeaway is that a few small changes to the Caesar cipher (a shift per position and a repeating key) give a much larger key space, $31^7 \approx 2.75 \cdot 10^{10}$ for the shortest allowed key, and hide the letter frequencies. The cipher is still breakable once the key length is found, because every l-th letter is then just a Caesar cipher.

5. SOURCE CODE

The full source code is available at `https://github.com/The1Dani/cs2026/tree/main/Lab3`.